RAPPORT PROJET – Backend API Blog

Prof : Mr GANSBEOGO
Etudiant: OUEDRAOGO Martin Hyacinthe

Node.js + Express + MySQL

1. Introduction

Ce projet ayant pour objectif de développer une API REST complète pour une application de blog moderne l''API permet de gérer des articles, des commentaires et des catégories, avec une authentification sécurisée via JWT. L'architecture respecte le modèle MVC (Modèle-Vue-Contrôleur) et les bonnes pratiques du développement backend.

2. Objectifs pédagogiques

Les objectifs atteints sont :

- Comprendre l'architecture d'une API REST moderne.
- Mettre en place un serveur Node.js avec Express.
- Connecter une base de données MySQL avec `mysql2/promise`.
- Implémenter des opérations CRUD complètes.
- Structurer un projet backend en MVC.
- Gérer les erreurs côté serveur.
- Tester une API avec REST Client (VS Code).
- Créer et utiliser un token JWT pour l'authentification (login/register).

---

3. Technologies utilisées

Technologie	Rôle
Node.js (v20)	Environnement d'exécution JavaScript
Express (4.18)	Framework web pour l'API
MySQL (8.0)	Base de données relationnelle
mysql2	Pilote MySQL avec support des promesses
dotenv	Gestion des variables d'environnement
cors	Middleware pour autoriser les requêtes cross-origin
bcryptjs	Hachage des mots de passe
jsonwebtoken	Génération et vérification des tokens JWT
nodemon	Redémarrage automatique du serveur en développement
REST Client (extension VS Code)	Tests des endpoints

4. Architecture du projet (MVC)

L’architecture MVC est rigoureusement respectée. Voici la structure des dossiers :

```
Back-End-MartinHyacinthe/
├── index.js              # Point d’entrée (démarre le serveur)
├── package.json
├── .env                  # Variables d’environnement
├── .gitignore
├── src/
│   ├── app.js            # Configuration Express (middlewares, routes)
│   ├── config/
│   │   └── db.js         # Connexion MySQL (pool)
│   ├── models/           # Accès aux données (requêtes SQL)
│   │   ├── userModel.js
│   │   ├── articleModel.js
│   │   ├── commentModel.js
│   │   └── categoryModel.js
│   ├── controllers/      # Logique métier
│   │   ├── authController.js
│   │   ├── articleController.js
│   │   ├── commentController.js
│   │   └── categoryController.js
│   ├── routes/           # Définition des endpoints
│   │   ├── auth.js
│   │   ├── articles.js
│   │   ├── comments.js
│   │   └── categories.js
│   └── middleware/       # Fonctions intermédiaires
│       ├── authMiddleware.js
│       └── errorMiddleware.js
├── database/
│   └── blog.sql          # Script de création de la base de données
└── tests/
    └── api.rest          # Fichier de tests REST Client
```

Rôle de chaque couche

- Modèles: contiennent les requêtes SQL (SELECT, INSERT, UPDATE, DELETE). Ils n’ont aucune logique métier.
- Contrôleurs : reçoivent la requête HTTP, valident les données, appellent les modèles, et renvoient la réponse JSON.
- Routes : associent une URL et une méthode HTTP à un contrôleur, et appliquent les middlewares (ex: authentification).
- Middleware : interceptent la requête pour vérifier le token JWT ou gérer les erreurs globalement.

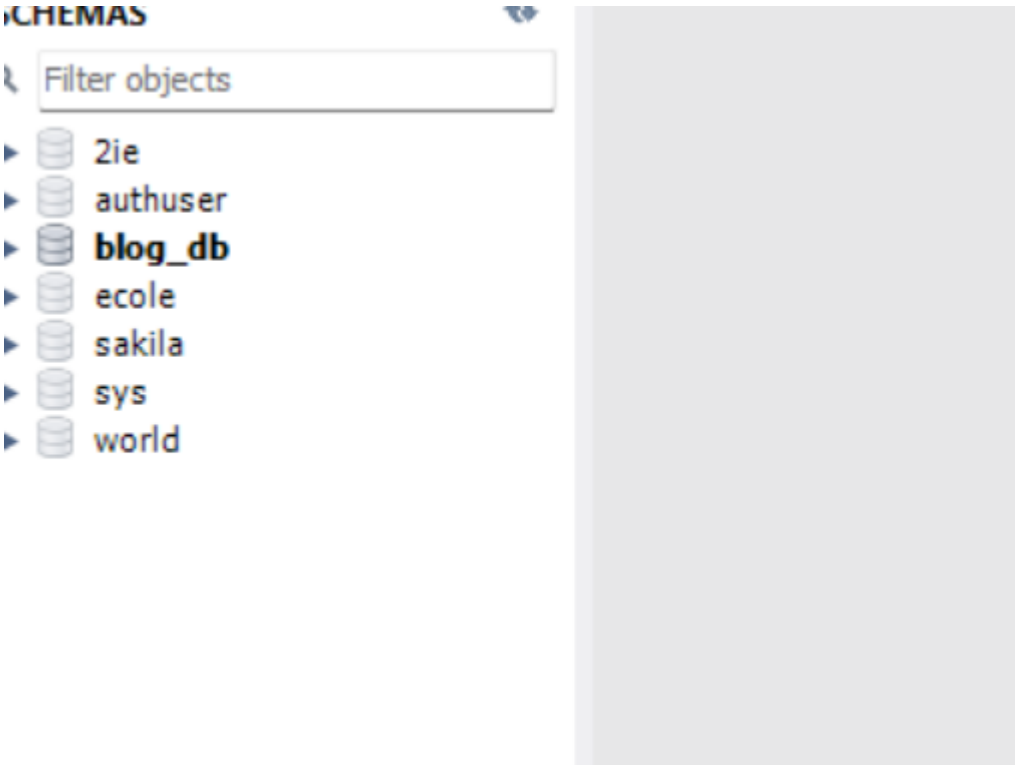
5. Configuration de l’environnement

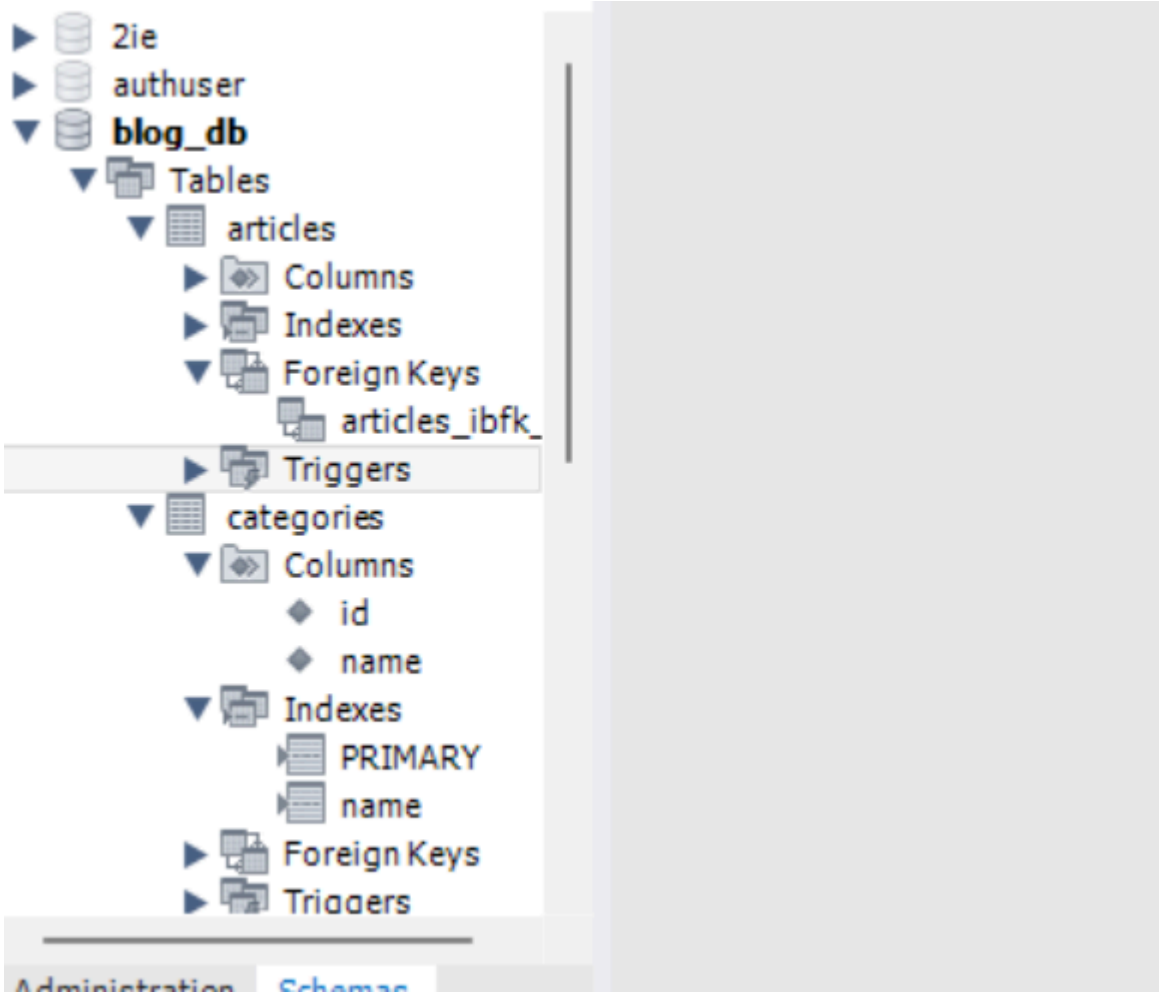
Fichier `.env` (exemple) :

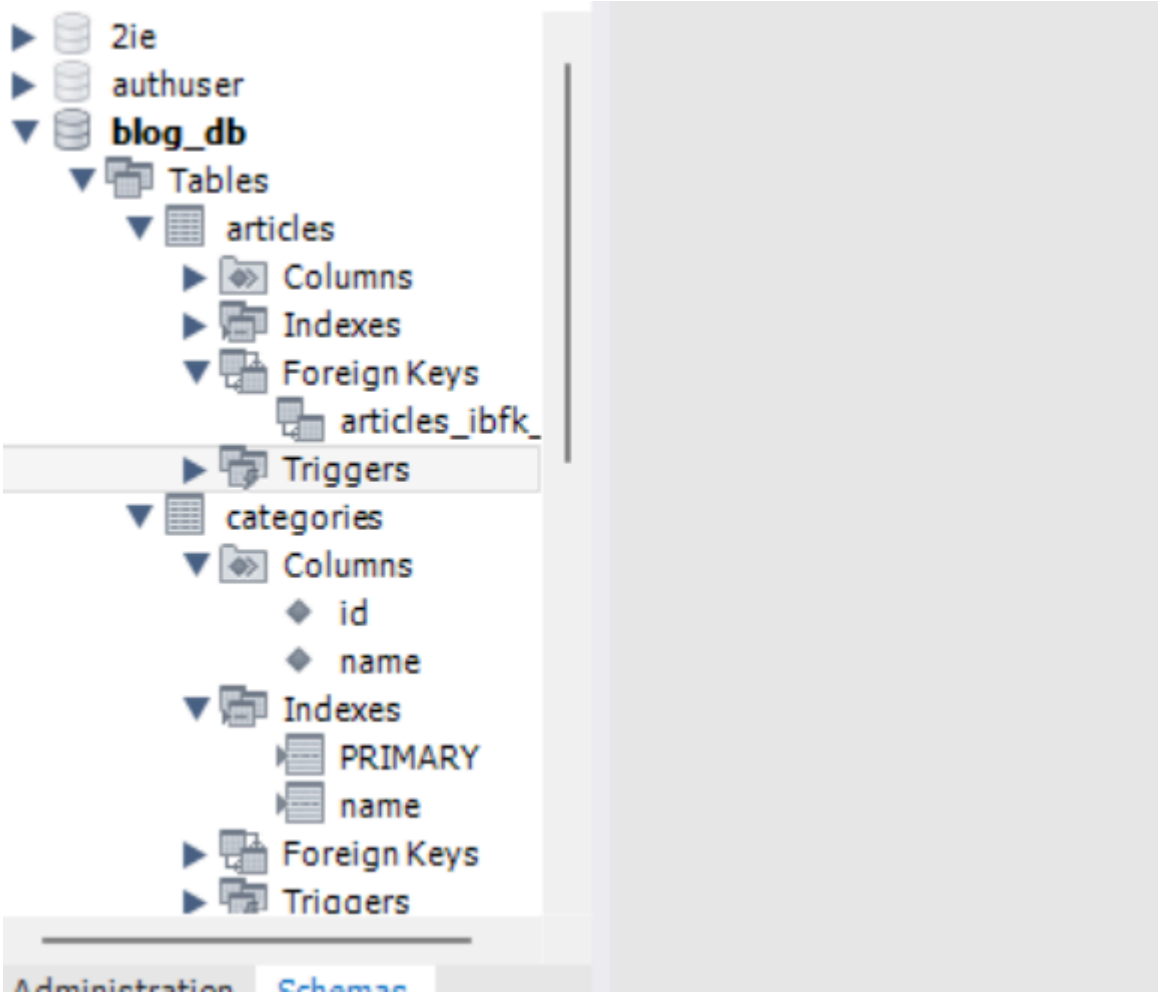
```env
PORT=5000
DB_HOST=localhost
DB_USER=root
DB_PASSWORD=root
DB_NAME=blog_db
JWT_SECRET=super_secret_key_change_me
JWT_EXPIRE=7d
```

6. Base de données (MySQL)

Schéma relationnel


SCHEMAS
Filter objects
2ie
authuser
blog_db
ecole
sakila
sys
world


2ie
authuser
blog_db
Tables
articles
Columns
Indexes
Foreign Keys
articles_ibfk_
Triggers
categories
Columns
id
name
Indexes
PRIMARY
name
Foreign Keys
Triggers
Administration
Schemas



Requêtes SQL principales

Le fichier `database/blog.sql` contient la création complète et des données de démonstration (admin, catégories, article, commentaire).

7. Implémentation de l'API

7.1. Endpoints disponibles

Méthode	Endpoint	Description	Authentification
POST	`/api/auth/register`	Inscription utilisateur	Non
POST	`/api/auth/login`	Connexion (retourne JWT)	Non
GET	`/api/articles`	Liste tous les articles	Non
GET	`/api/articles/:id`	Détail d'un article	Non
POST	`/api/articles`	Créer un article	Oui (JWT)
PUT	`/api/articles/:id`	Modifier un article	Oui (JWT)
DELETE	`/api/articles/:id`	Supprimer un article	Oui (JWT)

GET	`/api/comments/article/:id`	Commentaires d'un article	Non
POST	`/api/comments/article/:id`	Ajouter un commentaire	Non
GET	`/api/categories`	Toutes les catégories	Non
POST	`/api/categories`	Créer une catégorie	Oui (JWT)

8. Gestion des erreurs

Un middleware global capture toutes les erreurs et renvoie une réponse JSON standardisée :

```javascript
const errorMiddleware = (err, req, res, next) => {
  console.error(err.stack);
  const status = err.status || 500;
  const message = err.message || 'Erreur interne du serveur';
  res.status(status).json({ error: message });
};
```

9. Tests API avec REST Client

Le fichier `tests/api.rest` contient toutes les requêtes de test. Voici un exemple de séquence :

1. Inscription (`POST /api/auth/register`) – crée un utilisateur.
2. Connexion (`POST /api/auth/login`) – récupère un token JWT et le stocke automatiquement dans une variable `auth_token`.
3. Créer une catégorie** (`POST /api/catégories`) – utilise le token.
4. Créer un article** (`POST /api/articles`) – utilise le token.
5. Récupérer tous les articles** (`GET /api/articles`) – public.
6. Ajouter un commentaire** (`POST /api/comments/article/1`) – public.
7. Modifier / Supprimer** (avec token).

10. Difficultés rencontrées et solutions

Problème	Solution
Erreur `MODULE_NOT_FOUND` sur `index.js`	Création manuelle du fichier `index.js` avec le bon contenu.
`auth_token is not found` dans REST Client	Exécution de la requête `login` avant les requêtes protégées, et suppression des lignes vides entre le corps et le script
PowerShell incompatible avec `rm -rf`	Utilisation des commandes `Remove-Item -Recurse -Force`.
Fichiers manquants dans `src/` (ex: `app.js`)	Recréation complète de la structure avec des commandes `Out-File`.
Base de données non initialisée	Exécution du script `blog.sql` dans MySQL.

---

11. Conclusion

Le projet m’a permis de mettre en œuvre une API REST complète, sécurisée et structurée selon l’architecture MVC. Toutes les fonctionnalités demandées sont présentes :

- CRUD articles, commentaires, catégories.
- Authentification JWT (inscription, connexion).
- Base de données MySQL normalisée avec relations.
- Middlewares de gestion d’erreurs et d’authentification.
- Tests API avec REST Client.